SessionBound: Database-Enforced Task-Bound Sessions for Enterprise AI Agents

Minmin Wu
China Telecom Global Limited
wuminmin@chinatelecomglobal.com

***Abstract*—Enterprise AI agents can write useful exploratory SQL for audit, compliance, and operational analysis, but raw database access is too broad and fixed SaaS screens are too rigid. SessionBound bridges this gap by turning an approved business task into a short-lived, budgeted, and auditable database session. A control plane defines task templates, applications, approvals, TTLs, budgets, and signed task tokens; a database runtime, SessionBoundDB, binds the token to an agent session and enforces safe views, row scope, denied fields, read-only SQL policy, query/disclosure budgets, and receipts.**

The prototype is implemented over PostgreSQL with API-layer AST preflight and an experimental parse/analyze hook path. Evaluation validates 24 of 24 canonical scenarios and a 28-case adversarial SQL suite with 22 blocked cases, 5 allowed safe-view analytical cases, and 1 known limitation. On the default synthetic seed, full SessionBound p50 latency is 14.6–18.6 ms across representative query patterns; a 100k-row scale sweep exposes multi-second latency in the PL/pgSQL wrapper path, while hook-only structural checks are 0.130–0.169 ms p50, indicating a wrapper materialization/accounting bottleneck rather than parse/analyze guard cost.

SessionBound is not a replacement for RLS, ABAC, delegation, data catalogs, or differential privacy. It composes database-side enforcement with a task-control-plane contract: business users approve tasks, agents generate SQL, and the database decides whether each attempt stays inside the approved boundary. Code, synthetic data, validation reports, benchmark outputs, and LaTeX source are available at https://github.com/SessionBound/sessionbound.

***Index Terms*—AI agents, access control, database security, dependable computing, secure computing, authorization, SQL security, data governance, auditability.**

I. Introduction

AI agents change how enterprise users interact with data. A user no longer needs to know which report exists, which dashboard to open, or which API endpoint to call. Instead, the user can ask an agent:

> Analyze June 2026 travel reimbursements for the Sales department. Find unusual merchants, repeated claims, employee-level monthly totals, and claims near approval thresholds. Exclude salary and bank data.

This is exactly the kind of task agents are good at: temporary, exploratory, cross-table, and under-specified. It is also exactly the kind of task traditional enterprise software handles poorly. Building a permanent SaaS page or API for every low-frequency internal analysis task is expensive. Giving the agent raw database access is unsafe. Relying on prompts is not a security boundary. Relying on the application layer alone becomes fragile when SQL is generated dynamically.

Existing systems cover important pieces: delegated identity and OAuth scopes [1], [2], task-scoped tool authorization [3], data-product governance [4], [5], database-native policy enforcement [6], [7], [8], and large-scale relationship authorization [9]. They do not, by themselves, turn one approved analytical task into a temporary SQL session with safe views, denied fields, budgets, and receipts. That is the gap SessionBound addresses.

This paper argues for the following principle: business users approve tasks, not database policies. Agents generate SQL, but databases enforce the approved boundary. This paper frames the problem as a database-enforced authorization boundary for enterprise AI agents, rather than as an agent-planning or multi-agent coordination problem.

SessionBound separates three concerns.

1. Task control plane. Defines task templates, accepts task applications, evaluates grants and approvals, assigns budgets and TTLs, and signs task tokens.
2. Agent execution. Lets an LLM-powered agent decide what SQL to try, which hypotheses to test, and how to construct a temporary workspace.
3. Database runtime. Binds the signed task token to a database session and enforces safe views, scope, denied fields, operation limits, query budgets, disclosure budgets, and receipts.

The central design rule is that agents may explore only within a structured boundary that the database enforces deterministically.

This makes SessionBound a contract layer between enterprise approval and agentic database execution. The control plane records what the organization approved; the runtime enforces how that approval may be spent through SQL. This contract is not a database policy written by a DBA,

nor a natural-language instruction interpreted by an LLM. It is a structured, signed, and accountable representation of an approved business task that both the agent runtime and the database can interpret deterministically.

SessionBoundDB is our PostgreSQL-based reference runtime for SessionBound. It does not ask an LLM whether a query is safe. It does not depend on the agent correctly interpreting the task. It does not trust mutable session variables set by the agent. Instead, it validates a signed task token, exposes only registered safe views, rejects raw table access and denied fields, tracks query and disclosure budgets, and records receipts for evaluated allowed executions and bound-runtime denial decisions.

For compatibility with the existing demo implementation, the PostgreSQL prototype currently keeps the SQL schema name `taskbound`.

The novelty is not any single primitive. It is the binding of task approval, credential identity, safe-view versioning, SQL surface, cumulative disclosure budget, and receipt semantics into one database-execution session object: enterprise task approval is compiled into an execution-time database session contract rather than scattered across separate credentials, views, policies, and audit logs.

A. Contributions

This paper makes five contributions.

1. Enterprise task approval as the missing layer. We identify a gap between agent delegation and database enforcement: real enterprises approve tasks, scopes, and budgets above the database, but need those approvals translated into database-enforceable execution boundaries.
2. SessionBound control plane. We propose a task control plane with task templates, task applications, task approvals, grants, TTLs, budgets, and signed task tokens. This control plane lets business and data-governance workflows define what work is authorized without requiring managers to write database policies.
3. Budgeted database sessions. We define a database session model in which an approved task token binds an agent to safe views, row scope, denied fields, allowed operations, TTL, query budget, disclosure budget, and audit receipt policy.
4. LLM-independent enforcement for open-ended SQL. SessionBoundDB allows agents to generate SQL freely inside a boundary, but the database enforces that boundary deterministically. The database does not parse the natural-language task and does not rely on an LLM to judge safety at query time.
5. PostgreSQL prototype and evaluation. We implement SessionBoundDB, validate canonical and adversarial SQL scenarios, measure overhead, and analyze the hardening path from the current wrapper/hook prototype to production planner or executor integration.

II. Motivation: Enterprise Tasks Are Approved Above the Database

SessionBound targets temporary enterprise analysis that is too specific for a permanent application screen and too sensitive for raw database access: reimbursement audits, vendor-concentration checks, compliance-log reviews, or operational investigations. These tasks need joins, aggregation, ranking, drill-down, and follow-up SQL, but the exact query plan is not known when the task is approved.

Consider one running example. A business user requests:

> **Task type:** monthly expense anomaly review
> **Scope:** June 2026, Sales department
> **Purpose:** travel reimbursement audit
> **Sensitivity:** exclude salary, phone, and bank account fields
> **Budget:** 20 SQL attempts, 5,000 result rows, bounded entity exposure
> **TTL:** 30 minutes

A manager or data owner can approve this task without writing RLS predicates or database grants. A data platform can expose safe views. A compliance policy can tighten budgets. The agent can then generate SQL inside the approved boundary, while the database rejects raw tables, denied fields, unsafe operations, and over-budget attempts.

```
Business users approve tasks.
Data platforms register safe views.
Control planes sign task tokens.
Databases enforce boundaries.
Agents explore freely inside
those boundaries.
```

III. Threat Model

SessionBound assumes the upper-layer agent is useful but not trusted. The agent may be confused, overconfident, prompt-injected, malicious, or simply wrong. The database boundary must remain meaningful even when the agent tries the wrong thing.

A. Trusted components

For this prototype, we trust:

- the task control plane that defines templates, evaluates grants, records approvals, and signs task tokens;
- the credential broker that issues short-lived database credentials;
- the database runtime that validates tokens and enforces policies;
- the cryptographic signing key or signing service used for task tokens;
- the integrity of registered safe views and runtime functions.

In production, the signing path should be backed by a KMS, JWKS, hardware-backed key service, or enterprise identity infrastructure. The prototype uses a simpler signing path for demonstration.

B. Untrusted or partially trusted components

We do not trust:

- the agent's prompt;
- the agent's reasoning;
- the agent's SQL generation;
- the agent's natural-language interpretation of the task;
- user-provided or database-provided text that may contain prompt injection;
- ordinary client-controlled session variables;
- application-layer filtering as the final boundary.

This is the main difference from designs that require an LLM to infer the precise intended operation from natural language. SessionBound does not ask the database to understand the user's natural-language intent. The database consumes structured claims from a signed token.

C. In-scope threats

SessionBoundDB is designed to mitigate:

- Sensitive field access: attempts to read salary, bank account, phone, identity number, credentials, or private notes.
- Raw table access: attempts to bypass safe views and query application schemas directly.
- Scope escape: attempts to read other tenants, months, departments, projects, or users.
- Unauthorized mutations: write, DDL, or other high-impact operations outside controlled commands.
- Prompt injection effects: data or user text instructs the agent to ignore policy or reveal secrets.
- Budget evasion: repeated queries, pagination, or alternate SQL forms attempt to disclose more detail rows than the task allows.
- Credential-token mismatch: attempts to combine one runtime credential with another task token.

D. Out of scope

The prototype does not solve:

- malicious database administrators;
- compromised database host operating systems;
- compromised signing infrastructure;
- side-channel attacks;
- complete formal verification of SQL equivalence;
- production-grade planner/executor-level SQL enforcement;
- cross-database federation;
- full differential privacy for statistical releases;
- arbitrary semantic inference across all allowed query answers.

SessionBound reduces and accounts for what an agent can discover. It does not claim to eliminate all inference.

IV. SessionBound Model

SessionBound has two major layers: a task control plane and a database runtime. Figure 1 shows the main SessionBound control-plane and runtime boundary.

A. Task template

A task template is defined by a data platform or application team. It describes a class of work that can be safely delegated to agents once scoped and approved.

Example:

```
{
  "task_type": "monthly_expense_anomaly_review",
  "purpose": "travel_reimbursement_audit",
  "allowed_views": ["expenses", "employees", "departments"],
  "denied_columns": [
    "employees.salary",
    "employees.bank_account",
    "employees.phone"
  ],
  "required_scope": ["tenant_id", "expense_month", "department_id"],
  "allowed_operations": ["SELECT"],
  "default_ttl_minutes": 30,
  "default_budget": {
    "max_queries": 20,
    "max_result_rows": 5000,
    "max_unique_entities": {
      "expense_id": 5000,
      "employee_id": 200
    }
  }
}
```

The template is not generated by the agent. It is part of enterprise governance.

B. Task application

A task application is a concrete request to perform work under a template.

```
{
  "task_type": "monthly_expense_anomaly_review",
  "requested_by": "user:alice",
  "actor": "agent:travel-expense-analyst",
  "scope": {
    "tenant_id": "company_a",
    "expense_month": "2026-06",
    "department_id": "dep_sales"
  },
  "reason": "Review unusual travel reimbursement patterns."
}
```

C. Task approval

A task approval evaluates whether the requester may run this task with this scope and budget. Approval may be automatic, role-based, or manual. For sensitive tasks, a manager, data owner, or compliance reviewer may approve it.

The approval does not require the approver to write SQL policies. It decides whether this scoped task should exist.

D. Task token

A signed task token is the structured authorization object consumed by the database runtime.

```
{
  "task_id": "task_456",
  "task_type": "monthly_expense_anomaly_review",
  "delegator": "user:alice",
  "actor": "agent:travel-expense-analyst",
  "credential_id": "cred_123",
  "tenant_id": "company_a",
  "purpose": "travel_reimbursement_audit",
  "allowed_views": ["expenses", "employees", "departments"],
  "denied_columns": [
    "employees.salary",
    "employees.bank_account",
```

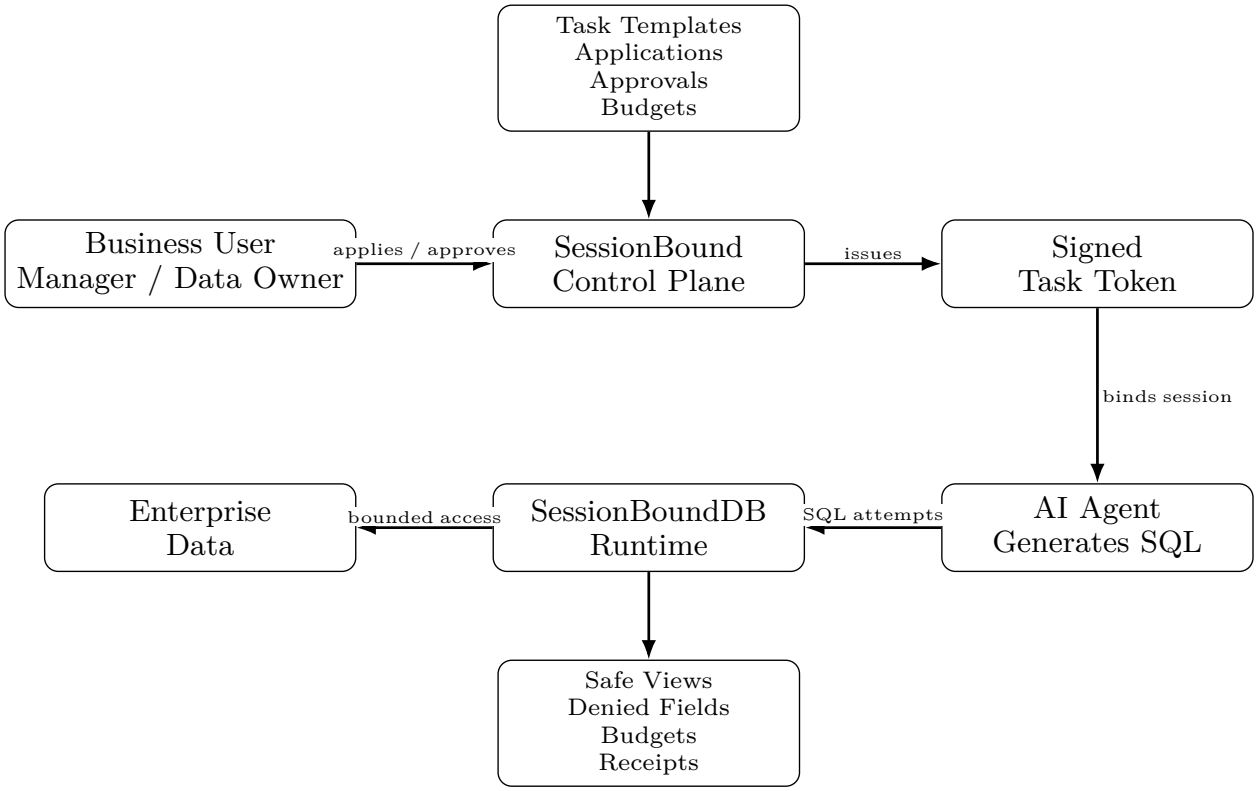

Fig. 1. SessionBound architecture. The control plane turns enterprise task approval into a signed task token; the agent may generate SQL freely, but SessionBoundDB enforces safe views, denied fields, budgets, and receipts inside the database session.

```
    "employees.phone"
  ],
  "row_scope": {
    "expense_month": "2026-06",
    "department_id": "dep_sales"
  },
  "operations": ["SELECT"],
  "budget": {
    "max_queries": 20,
    "max_result_rows": 5000,
    "max_unique_entities": {
      "expense_id": 5000,
      "employee_id": 200
    }
  },
  "safe_view_registry_version": "travel-demo-v1",
  "policy_version": "expense-review-v1",
  "audience": "sessionbounddb",
  "expires_at": "2026-06-24T10:30:00Z",
  "nonce": "..."
}
```

E. Budgeted database session

A budgeted database session is a database session bound to one active task token. The session may issue open-ended SQL, but every attempt is checked against the token, safe view registry, operation policy, and remaining budget.

In the current prototype, a direct database connection may issue native `SELECT` over approved safe views after binding a signed task token. The PostgreSQL extension checks SQL shape and safe-view OIDs, then executor accounting counts rows and disclosed `expense_id` values before tuples are released. The same path covers SDK `query(sql)`, bound direct `SELECT`, prepared statements, cursor/`FETCH`, `COPY (SELECT)`, and non-`ANALYZE` `EXPLAIN`. Raw table `SELECT` remains ungranted; schema resolution is allowed only so the hook can fail closed and emit receipts. Pooled connections can rebind only with a fresh valid token and can call `unbind_task()` at check-in.

V. Control Plane: Templates, Applications, Approvals, and Budgets

The control plane is the SaaS-like layer that makes SessionBound realistic for enterprises. This is where SessionBound differs most clearly from database-only security systems.

A. Business users approve tasks, not policies

A data owner should see:

```
Alice requests a June Sales
travel reimbursement anomaly review.
Scope: Sales department, June 2026.
Denied: salary, bank account, phone.
Budget: 20 queries, bounded
entity exposure.
TTL: 30 minutes.
```

The data owner should not see:

```
CREATE POLICY ...
ALTER ROLE ...
GRANT SELECT ON ...
```

The control plane converts the business approval into a signed token that the runtime can enforce.

B. Grants and eligibility

Roles do not grant direct database access. They grant eligibility to request task templates. For example, a finance reviewer may be eligible to request finance-review tasks; a department manager may be eligible to request department-scoped analysis tasks.

This avoids broad database roles such as:

```
finance_user can access all finance tables
```

Instead, the database sees a concrete task token: agent X may analyze June 2026 Sales reimbursements using views A, B, C under budget Y.

C. Budget assignment

Budget assignment is a control-plane decision. A low-risk task might receive a larger aggregate budget; a sensitive task may receive a small detail-row budget and a short TTL. Budgets can be adjusted by role, data classification, task purpose, and approval route.

The database runtime enforces the budget; the control plane decides what budget is appropriate.

VI. SessionBoundDB Runtime

SessionBoundDB is the PostgreSQL-based database runtime for SessionBound.

A. Session binding and credential-token binding

The runtime begins with two objects:

1. a short-lived database credential issued by the credential broker;
2. a signed task token issued by the control plane.

The credential authenticates the runtime connection. The task token authorizes the work.

At `bind_task`, the runtime checks:

- the task token signature is valid;
- the task token is unexpired and not revoked;
- the credential is unexpired and not revoked;
- the token `credential_id` matches the current session credential;
- the token `actor` matches the credential actor;
- the session login role matches the broker-issued database user;
- the token audience is `sessionbounddb`;
- no other active task is already bound to this database backend/session.

A production implementation should make a stolen credential without a matching token insufficient, and a stolen token without the matching credential insufficient. Credential-token binding is intended to reduce cross-task credential misuse.

PostgreSQL nuance: inside `SECURITY DEFINER` functions, `current_user` may refer to the function owner. Therefore, a production implementation should use `session_user` and/or an explicit credential registry mapping rather than relying on `current_user` alone. The measured prototype validates token signatures, token expiration, revoked task state, credential-id-to-token matching, actor matching, audience validation, cross-credential token replay rejection, and same-session rebind rejection in the tested prototype path; Section XIII reports those tests explicitly. Production deployments still require hardened credential brokerage, key management, and audit-retention integration outside this demo.

B. Safe views as the query surface

Agents do not query raw application tables. They query registered safe views. Safe views expose business objects, not internal schemas. This is complementary to database-native mechanisms such as PostgreSQL row-level security policies, which provide row filtering at the database layer [6].

Example safe views:

```
expenses
employees
departments
approval_events
ledger_entries
```

A safe view may:

- join raw tables into business objects;
- remove sensitive fields;
- apply tenant, department, month, or project scope;
- expose workflow hints;
- include stable business identifiers for budget accounting.

Safe views are not a complete inference-control solution. They reduce the discoverable surface and make it governable.

C. Operation policy

Native PostgreSQL `SELECT` is the database accounting entrypoint for read-only analytical SQL. The agent-facing SDK exposes this as `query(sql)`: the agent supplies ordinary SQL over approved safe-view names, and PostgreSQL hook and executor paths enforce the task boundary, debit query and disclosure budgets, and emit receipts. `taskbound.run(sql)` remains as a compatibility wrapper. High-value writes should use controlled commands or stored procedures. The runtime rejects mutations, DDL, unsafe utility commands, raw schema access, denied fields, and blocked schemas.

D. Query decision pipeline

At runtime, every SQL attempt follows the deterministic decision procedure in Algorithm 1.

Algorithm 1. SQL decision pipeline for SessionBoundDB
Input: SQL attempt and bound database session state.
Output: Allowed result with receipt, or denial receipt.

1) Require an active task binding for the session.
2) Validate the signed task token and its TTL.
3) Resolve all referenced relations against registered safe views.
4) Reject denied fields and blocked schemas.
5) Enforce row scope predicates for the bound task.
6) Reject disallowed operation types, including mutations, DDL, and unsafe utility commands.
7) Check remaining query and disclosure budgets.
8) If every check passes, execute the query and emit an allow receipt; otherwise reject the attempt and emit a denial receipt.

The runtime does not ask an LLM whether a query is safe.

VII. Security Invariants

The SessionBoundDB enforcement contract can be described by a state

$$S = \langle T, C, V, B, R \rangle, \tag{1}$$

where T is the signed task token, C is the credential/session binding, V is the safe-view registry and policy version, B is the remaining budget vector, and R is the receipt chain. Each SQL attempt is evaluated by:

$$\begin{aligned} \mathit{decide}(q, S) \rightarrow\ & \mathit{allow}(\mathit{result}, S') \\ & \text{or } \mathit{deny}(\mathit{reason}, \mathit{receipt}, S'). \end{aligned} \tag{2}$$

These invariants define the enforcement contract implemented by the prototype and used to structure the adversarial evaluation. They are not a formal proof that all semantic inference is impossible.

- I1. No SQL execution occurs without an active task binding.
- I2. The bound token must match the session credential, actor, audience, expiry, token digest, and revocation state.
- I3. Referenced relations must belong to the approved safe-view registry.
- I4. Direct access to denied fields, raw schemas, catalog escape, mutation, DDL, and blocked payload aggregation is denied.
- I5. Scope predicates or session-bound claims constrain visible rows.
- I6. Budget state is monotonic: an allowed query consumes budget, or the query is denied.
- I7. Every allow/deny decision emits a receipt.
- I8. Safe-view registry, policy version, or definition-hash drift invalidates the token or requires reapproval.

In the prototype, I1–I2 and I8 are checked during `taskbound.bind_task`; I3–I4 are checked by API-layer AST preflight, the experimental `sessionbound_guard` PostgreSQL hook path, and database permissions; I5 is enforced by safe-view predicates; I6 is implemented through task execution state; and I7 is implemented through allow/deny receipts. Production implementations should harden these structural checks into always-on planner or executor integration, but the invariant set is independent of that implementation choice.

A. Security Guarantees for the Prototype SQL Fragment

We state the prototype security contract over a deliberately restricted SQL fragment. Let SELECT-F denote single-statement, read-only SQL consisting of `SELECT`, projection, predicates, joins, `GROUP BY`/`HAVING`, ordering and limits, non-recursive CTEs, and window functions over registered safe-view names. The fragment excludes stacked statements, DDL, DML, `COPY`, `DO`, `CALL`, `CREATE FUNCTION`, temporary object creation, recursive CTEs, set-operation stacking, table functions, raw-schema or catalog references, and high-risk payload aggregation such as `json_agg`, `jsonb_agg`, `array_agg`, `string_agg`, `xmlagg`, `row_to_json`, `json_build_object`, and `jsonb_build_object`.

Let $\mathrm{Rels}(q)$ be the set of relation identifiers in q after database name resolution and before safe-view expansion. Let $\mathrm{Views}(T, V)$ be the approved safe views carried by token T and validated against registry V. Let $\mathrm{Cols}_{out}(q)$ be the direct column references that appear in the output expressions of q. A state $S = \langle T, C, V, B, R \rangle$ is well formed when the token is valid and bound to credential C, the safe-view registry and policy hashes match the token, the budget vector B is nonnegative, and the receipt chain R verifies. Under these assumptions, the prototype provides the following direct-reference and accounting guarantees. These are not guarantees against arbitrary semantic inference from allowed answers, and they are not differential privacy.

Proposition 1 (Safe-surface confinement). For any well-formed state S and query $q \in$ SELECT-F, if $\mathit{decide}(q, S) = \mathit{allow}(\mathit{result}, S')$, then $\mathrm{Rels}(q) \subseteq \mathrm{Views}(T, V)$. Consequently, every raw base relation evaluated during query execution is reached only through the definition of a registered safe view approved for the task; the query has no direct reference path to raw tables or catalogs.

Argument. The decision pipeline resolves relation identifiers and checks them against the task's approved safe-view registry before any result is released. Database permissions deny raw-schema access, while the AST preflight and hook path reject resolved relation OIDs outside the approved safe-view set. Therefore any query that names an unregistered relation is denied. Raw base tables may still be read by trusted safe-view definitions, but not by direct agent-supplied SQL.

Proposition 2 (Denied-field non-disclosure for direct references). Let $D(T)$ be the denied-field set in token T, and let $\mathrm{Cols}(v, V)$ be the registered column list for safe view v. If $\mathit{decide}(q, S) = \mathit{allow}(\mathit{result}, S')$, then every direct output column reference $c \in \mathrm{Cols}_{out}(q)$ is in $\bigcup_{v \in \mathrm{Views}(T,V)} \mathrm{Cols}(v, V)$ and $c \notin D(T)$. Thus a column that is absent from the approved safe-view column lists, or that appears in the denied-field set, cannot be directly projected by an allowed query.

Argument. In SELECT-F, output columns must resolve against the relations that survived Proposition 1. A column not exported by an approved safe view cannot be bound by name. A column name or alias in the denied-field set is rejected by the structural policy before execution. The guarantee is syntactic and direct: it does not rule out inferences about denied fields from permitted columns or aggregates.

Proposition 3 (Monotonic budget accounting). Let budgets be ordered componentwise by $\preceq$, and let $\Delta(q, \mathit{result}) \succeq 0$ be the budget charge for a released result. For any allowed query, $\mathit{decide}(q, S) = \mathit{allow}(\mathit{result}, S')$ implies $0 \preceq B' \preceq B$, where B' is the budget component of S'. If a required charge is not covered by the remaining budget, then the query is denied and no result is released.

Argument. The runtime treats budget dimensions as nonnegative counters. Query-count budget is checked before execution, and disclosure budget is charged before release using the accounted result shape and business identifiers. Updates only subtract nonnegative charges from the remaining vector. Therefore the remaining budget cannot increase across an allowed transition; an over-budget attempt reaches a denial state instead of returning the result.

Proposition 4 (Receipt-chain tamper evidence under trusted DB assumptions). Assume collision-resistant hashing and append-only receipt storage inside the trusted database boundary. For a receipt chain $R = (r_1, \ldots, r_n)$ where each r_i stores the hash of r_{i-1} and a hash of its own canonical contents, removing or modifying any interior receipt r_j, $1 < j < n$, changes r_j's hash or breaks the previous-hash value stored in r_{j+1}. The change is detected by chain verification unless the adversary can violate append-only storage or find a hash collision.

Argument. Each receipt commits to the previous receipt hash and to the current decision, query hash, timestamp, and budget delta. Changing an interior receipt changes its hash; deleting it changes the predecessor expected by the next receipt. Under append-only storage, the adversary cannot rewrite the downstream chain to hide the mismatch. This is a tamper-evidence property for the audit trail, not a Byzantine storage guarantee against malicious DBAs or compromised hosts.

VIII. Anti-Evasion Scope

SessionBound handles direct boundary violations and accounts for bounded disclosure; arbitrary semantic inference remains outside its formal guarantees.

A. Direct boundary violations

SessionBoundDB can directly block:

- access to raw schemas;
- access to sensitive fields that are not exposed through safe views;
- queries outside row scope;
- unsupported operation types;
- repeated querying beyond task budgets;
- use of credentials without a matching task token;
- attempts to continue after token expiry or revocation.

B. Inference and side channels

SessionBound does not claim to solve arbitrary inference from allowed answers. For example, if a safe view exposes enough correlated information, an agent may infer sensitive facts indirectly. Aggregates over small groups may leak more than intended. Timing and resource side channels are also outside the prototype's formal guarantees. Classical database inference-control work studies these risks for statistical databases and repeated query answers [10].

C. Adversarial SQL patterns

Table I summarizes representative evasion patterns, defenses, and limitations.

This table defines the prototype's anti-evasion scope and the path toward production hardening.

IX. Budget Semantics

SessionBound disclosure budgets limit how much task-authorized information an agent may retrieve during one approved analytical session. They are authority accounting, not differential-privacy accounting [11].

A production deployment should use a vector rather than a single unique-row limit. Useful dimensions include query count, result rows, result bytes, per-query payload bytes, aggregate payload shape, unique business entities, column weights, detail-versus-aggregate multipliers, and small-group penalties.

The prototype's unique-row accounting demonstrates the mechanism, while the broader model treats budget as a vector of exposure counters tied to task authority.

Budget vectors are not mutual-information bounds. They provide operational containment, auditability, and a governance lever for approved enterprise tasks.

TABLE I
ADVERSARIAL SQL PATTERNS AND SESSIONBOUND DEFENSES.

Pattern	Example	Defense	Limitation
Raw table escape	`SELECT * FROM app_data.expenses`	no raw grants; safe views only	strong in prototype
Sensitive column	`SELECT salary FROM employees`	denied fields; safe view exclusion	direct leakage only
Scope escape	query another month or department	safe-view predicates from task claims	strong if views are correct
Mutation	`DELETE`, `UPDATE`, `INSERT`	operation policy; read-only `run` path	writes require controlled commands
DDL	`DROP`, `ALTER`, `CREATE`	blocked operations	production should use hook-backed validation
Catalog escape	`pg_catalog`, `information_schema`	blocked schemas and no grants	production needs planner/executor hooks
Pagination scraping	repeated `LIMIT/OFFSET`	query budget and disclosure budget	budget model matters
JSON / array payload aggregation	payload aggregation functions	result-byte budget, per-query payload cap, optional aggregate allowlist / AST policy	production should detect high-risk aggregation functions
Aggregate inference	small-group `GROUP BY`	optional minimum group size and aggregate policy	future work
Timing side channel	expensive predicates	statement timeout and resource budget	partial

X. SCHEMA DRIFT AND SAFE VIEW VERSIONING

Task tokens should not silently survive changes to the data boundary they approve. If a safe view changes after approval, the token may no longer represent the approved task.

A task token should therefore bind safe views by more than name: registry version, policy version, database object identifier, view definition hash, and exposed-column hash. PostgreSQL views make this important: `CREATE OR REPLACE VIEW` may preserve `pg_class.oid` while changing the definition, and `relfilenode` is not an appropriate view identity. A task-bound session should fail closed if its registry version, policy version, view definition hash, or exposed-column hash no longer matches the approved boundary.

XI. RECEIPTS

SessionBound receipts are structured audit records that bind task identity, query decision, and budget impact. They are not merely logs.

Each receipt records the task id, actor, query hash or command name, decision, touched safe views, rows returned, budget delta, remaining budget, timestamp, previous receipt hash, and current receipt hash.

The prototype implements a lightweight hash chain: each receipt stores `previous_receipt_hash` and `receipt_hash`, where the current hash commits to the task id, query hash, decision, budget delta, timestamp, and previous hash. This does not provide a full external proof system, but it makes post-hoc tampering with the receipt sequence detectable within the database audit trail.

XII. PROTOTYPE IMPLEMENTATION AND PRODUCTION HARDENING

A. Prototype

The current prototype includes:

- FastAPI control plane;
- task template and grant registry;
- short-lived PostgreSQL credential broker;
- signed task token issuance;
- PostgreSQL runtime functions;
- safe views over travel reimbursement data;
- `taskbound.bind_task(payload, signature)`;
- an agent SDK with `TaskboundSession.query(sql)`;
- native guarded safe-view `SELECT` plus compatibility `taskbound.run(sql)`;
- sensitive field denial;
- blocked raw table access;
- query and disclosure budget state;
- receipts for evaluated allowed executions and bound-runtime denial decisions;
- controlled commands for high-value workflow actions;
- an agent-agnostic HTTP evaluation harness.

The prototype demonstrates the SessionBound architecture, not a production-grade SQL safety proof.

B. Implementation honesty

The current hardening prototype uses API-layer AST preflight, PostgreSQL hooks, executor result accounting, rollback-surviving audit writes, and PostgreSQL permissions. Agents do not receive raw table `SELECT` grants. The SDK's `query(sql)` executes native SQL over the bound safe-view schema; a direct `SELECT ... FROM taskbound.expenses` is allowed only after binding and is accounted before rows are released. High-value writes go through controlled commands.

The database hook path is implemented by the `sessionbound_guard` C extension, loaded through `shared_preload_libraries`. The extension registers `post_parse_analyze_hook`, `ProcessUtility_hook`, and executor hooks. Trusted SUSET GUCs populated by `taskbound.bind_task` carry the task, budgets, receipt flags, and approved safe-view OIDs. The executor path wraps the destination receiver to count rows and disclosed `expense_id` values before forwarding tuples. Receipt and

budget updates use an autonomous same-database audit channel, so receipts for evaluated allowed executions and hook/API denials survive agent transaction rollback.

The hook rejects non-`SELECT` statements, raw `app_data` relations, catalog access, recursive CTEs, `UNION/INTERSECT/EXCEPT`, unapproved relation OIDs, non-view relations, unsafe non-catalog functions, sensitive output aliases, and high-risk payload aggregation functions such as `json_agg`, `jsonb_agg`, `array_agg`, `string_agg`, `xmlagg`, `row_to_json`, `json_build_object`, and `jsonb_build_object`. The API AST validator remains defense in depth and provides portable preflight feedback before the database runtime is invoked.

This should still be read as an experimental hook path, not a production SQL-safety proof. A production implementation should harden the same structural policy into:

- always-on PostgreSQL parser/analyzer hooks;
- planner hooks;
- executor hooks;
- extension-level session state and registry management;
- deny-by-default schema grants;
- blocked unsafe utility commands;
- executor-level result accounting for deployments that allow native agent `SELECT` syntax, including query-budget debit before execution, bounded result materialization or streaming interception, disclosure-budget accounting before client release, and receipt emission through an audit path that survives transaction aborts;
- `statement_timeout` and resource limits;
- cleanup for expired short-lived roles;
- denial logging outside user transactions.

The measured AST prototype extracts referenced relations, columns, function calls, CTEs, recursive CTEs, subqueries, joins, catalog access, operation type, and set operations. It blocks raw schema references, catalog access, mutations, DDL, `COPY`, `SET`, `SHOW`, `CREATE FUNCTION`, `DO` blocks, recursive CTEs, `UNION/INTERSECT/EXCEPT` stacking, unknown functions, denied-field aliases, and high-risk payload aggregation functions. The API AST validation evaluation passed 17 of 17 query-shape cases, and the PostgreSQL hook evaluation passed 11 of 11 direct database enforcement cases.

C. Query example

Allowed:

```
WITH ranked AS (
  SELECT expense_id, department_name, category, amount,
         row_number() OVER (
           PARTITION BY department_name ORDER BY amount DESC
         ) AS rn
  FROM expenses
  WHERE expense_month = '2026-06'
)
SELECT department_name, expense_id, category, amount
FROM ranked
WHERE rn = 1;
```

Denied:

```
SELECT employee_name, salary FROM employees;
```

Denied:

```
SELECT * FROM app_data.expenses;
```

Denied:

```
DELETE FROM expenses WHERE expense_month = '2026-06';
```

XIII. Evaluation

The prototype evaluation has two goals:

1. validate that approved analytical queries can run over safe views;
2. validate that direct boundary violations are denied and accounted for.

It does not prove production-grade SQL safety or eliminate all inference attacks. The evaluation focuses on functional boundary validation, security-property comparisons against narrower database controls, database-runtime overhead, and concurrency behavior on the default synthetic dataset.

A. Experimental setup

The latest hardening run used Docker Compose with PostgreSQL 16.14, the FastAPI SessionBoundDB prototype, and the `sessionbound_guard` extension loaded through `shared_preload_libraries`. The repository branch was `tdsc-hardening`. The default seed contains 430 expenses, 240 employees, and 124 departments. The raw outputs are stored under `paper/tdsc/raw_results/`.

The AST and adversarial SQL evaluations exercise the HTTP API, including dynamic credential issuance, signed task-token creation, token binding, AST preflight, the SDK-backed query path, budget state, and receipts. A separate SDK surface evaluation calls `TaskboundSession.query(sql)` directly from the Compose network. The hook evaluation bypasses the API query endpoint for agent cases by using generated runtime credentials directly against PostgreSQL, and it uses trusted superuser setup for hook-only structural checks. The overhead breakdown uses direct PostgreSQL connections from the Compose network so that it isolates database runtime overhead from HTTP, model calls, dynamic credential creation, and API-layer AST parsing. A hook-only microbenchmark measures `sessionbound_guard_check(sql)` directly, exercising PostgreSQL parse/analyze and the structural guard without executing result rows or performing wrapper materialization.

B. Functional validation

We evaluated the SessionBoundDB PostgreSQL prototype using Docker Compose. The public repository contains the evaluation scripts, validation reports, benchmark outputs, and source code needed to reproduce the results. The canonical validation suite passed 24 of 24 scenarios; the full scenario matrix and raw receipts are provided in the artifact repository rather than reproduced in the main text.

Scope violations over safe views are enforced by filtering rather than explicit denial. A query for another approved-view but out-of-scope month returns zero rows because the safe view predicate binds the session to the task scope. The measured prototype conservatively blocks payload aggregation functions such as `json_agg`, `array_agg`, `string_agg`, and `row_to_json`; ordinary aggregate analysis such as `count`, `sum`, `avg`, `min`, and `max` remains allowed.

C. Baseline security comparison

We compare SessionBound against four narrower database configurations to separate access-surface reduction from task-bound session enforcement:

- **Raw PostgreSQL**: direct access to application tables using a trusted analyst/admin test role.
- **Role-only**: a constrained database role with coarse grants but no task token, budget vector, receipt chain, or task-specific safe view binding.
- **RLS-only**: PostgreSQL row-level security policies over raw tables, without SessionBound task approval, disclosure accounting, or receipt semantics.
- **Safe-view-only**: curated views and denied-field exclusion, without credential-token binding, query budgets, disclosure budgets, or signed task tokens.
- **SessionBound full**: signed task token, short-lived credential binding, safe views, denied fields, operation policy, query budget, disclosure budget, anti-payload aggregation checks, and receipts.

Table IV summarizes the observed security properties. In the updated credential-token test suite, SessionBound denies credential-id mismatch, cross-credential token replay, wrong audience, wrong actor, expired tokens, revoked tasks, and same-session rebind to a second active task. It also rejects tokens whose safe-view registry version, policy version, or view-definition hash no longer matches the runtime registry.

D. Adversarial SQL evaluation

The latest hardening run adds an adversarial SQL suite with 28 cases across direct boundary violations, catalog escape, payload aggregation, obfuscation, CTEs, set-operation stacking, small-group aggregate inference, and search-path/function abuse. The suite passed all expected classifications: 22 cases were blocked, 5 safe-view analytical cases were allowed and accounted for, and 1 small-group aggregate query was classified as a known limitation.

This result should not be read as a proof of complete SQL safety. It does show that the current prototype is no longer limited to literal string checks for the main tested attack families. The remaining small-group aggregate case motivates minimum group-size rules, aggregate sensitivity scoring, and stronger inference-control work.

We separately evaluated the native SDK surface. A smoke test confirmed that `TaskboundSession.query(sql)` and bound direct `SELECT ... FROM taskbound.expenses` are both accounted, while the same query after `unbind_task()` fails closed.

We also evaluated the database hook path directly. The `sessionbound_guard` suite passed 16 of 16 cases. It confirmed GUC tamper resistance, allowed bound native `SELECT`, support for prepared statements, cursor/`FETCH`, `COPY (SELECT)`, and non-`ANALYZE EXPLAIN`, and blocked set operations, catalog/raw access, runtime-helper abuse, payload aggregation, unapproved OIDs, and `EXPLAIN ANALYZE`.

E. Performance overhead and ablation

The latest overhead breakdown measures five query patterns: simple `SELECT`, `JOIN`, `GROUP BY`, CTE, and window function. Each pattern uses 10 warmup iterations and 100 measured iterations per mode. The run compares raw PostgreSQL, a temporary role-only read-only credential, safe-view-only execution, unsupported RLS-only mode, SessionBound without receipts, SessionBound without budget updates, and full SessionBound. Table V reports p50 latency in milliseconds.

The RLS-only row is marked `N/A` in this run because the current prototype does not define PostgreSQL RLS policies. Earlier experiments separately measured an RLS baseline, but the latest breakdown focuses on currently supported ablations and does not invent an RLS toggle.

The main observation is that most overhead is already present in the safe-view-only mode. Raw and role-only queries are sub-millisecond, while safe-view-only p50 is 13.3–18.1 ms. Full SessionBound p50 is 14.6–18.6 ms. Disabling receipts or budget accounting does not consistently reduce latency, so receipt insertion and budget updates do not dominate this small-dataset benchmark. This is an important interpretation point: for the default seed, SessionBound's measured latency is mainly a safe-view/session-runtime cost rather than a receipt-chain cost.

1) Performance diagnosis: The database-runtime breakdown isolates direct PostgreSQL execution and therefore does not include HTTP latency, model calls, dynamic credential creation, or API-layer AST preflight. End-to-end agent requests do incur the AST preflight before entering PostgreSQL, so the end-to-end path is strictly larger than the database-only numbers in Table V.

Within the historical measured database path, the overhead comes from several implementation layers in the wrapper reference prototype. First, `taskbound.run` dispatches each query through a PL/pgSQL wrapper rather than through PostgreSQL's native planner and executor path alone. Second, each execution repeatedly resolves session claims and task state instead of caching a bound task descriptor in trusted backend state. Third, safe views add predicate and view-expansion cost even before SessionBound receipt or budget logic runs. Fourth, the wrapper path materializes result rows so that the runtime can account for returned business identifiers. Fifth, receipt insertion and budget

TABLE II
ADVERSARIAL SQL EVALUATION SUMMARY FROM THE HARDENING SUITE.

Category	Representative pattern	Result	Notes
Direct boundary violations	denied columns, raw schema, DML, DDL	5 blocked / 5	salary, bank account, `app_data`, `DELETE`, and `DROP` denied
Catalog and metadata escape	`pg_catalog`, `information_schema`, bare `pg_tables`	3 blocked / 3	catalog schemas and known catalog relations denied
Payload aggregation / compression	`json_agg`, `array_agg`, `string_agg`, `row_to_json`	6 blocked / 6	high-risk one-row payload compression denied
Obfuscation and aliases	harmless aliasing, denied-field alias	3 allowed, 1 blocked	allowed queries emitted receipts; `amount AS salary` blocked
Subqueries and CTEs	nonrecursive CTEs and recursive CTE	2 allowed, 1 blocked	ordinary CTEs allowed; recursive/set-operation shape denied
`UNION` and stacking	`UNION`, `UNION ALL`	2 blocked / 2	set-operation stacking denied
Small-group aggregate inference	group by department and employee	1 known limitation	allowed aggregate; no minimum group-size policy yet
Search path / function abuse	`SHOW`, `SET`, `CREATE FUNCTION`, `DO`	4 blocked / 4	unsafe utility/procedural forms denied

TABLE III
POSTGRESQL HOOK AND EXECUTOR ENFORCEMENT EVALUATION. AGENT CASES CONNECT DIRECTLY AS THE GENERATED RUNTIME CREDENTIAL AND ISSUE NATIVE SQL AFTER BINDING A SIGNED TASK TOKEN.

Case group	Enforcement path	Result	Notes
Trusted context	Non-superuser direct DB session	1 blocked / 1	Ordinary agent role cannot set SUSET guard GUCs
Native allowed surface	Agent credential direct DB session	6 allowed / 6	Bound `SELECT`, schema-qualified safe-view query, prepared `EXECUTE`, cursor/`FETCH`, `COPY (SELECT)`, and non-`ANALYZE EXPLAIN` accounted
Native blocked surface	Agent credential direct DB session	5 blocked / 5	`UNION`, catalog access, recursive CTE, private helper access, and `EXPLAIN ANALYZE` denied
Hook-only structural checks	Trusted-GUC hook check without API preflight	4 blocked / 4	Non-`SELECT`, raw schema, payload aggregation, and unapproved safe-view OID denied

accounting add state updates, but the no-receipt and no-budget ablations show that these updates are not the dominant source on the small default dataset.

To separate structural enforcement from wrapper-era execution cost, we also ran a hook-only microbenchmark over the native `sessionbound_guard_check(sql)` path. This path prepares SQL through PostgreSQL parse/analyze and the structural guard, but does not execute result rows, perform executor row accounting, insert receipts, or materialize JSON rows through `taskbound.run`. With 20 warmup and 200 measured iterations per query shape, allowed structural checks had p50 latency of 0.138 ms for SELECT, 0.169 ms for JOIN, 0.139 ms for GROUP BY, and 0.130 ms for CTE/window SQL; raw-schema and UNION denial checks also passed. This is not an end-to-end performance claim for the native executor-accounting path, but it shows that structural parse/analyze guarding is not the source of the 100k-row multi-second behavior.

The historical scale results in Table VI show a different regime. At 100k scoped expense rows, safe-view-only execution remains in the tens of milliseconds, while full SessionBound reaches 6.7–7.0 s. This implicates the PL/pgSQL reference runtime's row materialization and per-query accounting path, not safe-view predicates alone. We therefore interpret the prototype as a security reference implementation, not an optimized execution engine. A production implementation should bind the task token once, keep approved safe-view identifiers in trusted backend state, and move structural checks and disclosure accounting into parser/analyzer, planner, or executor hooks. The native hook/executor implementation described above addresses the structural-enforcement part of this path, but native executor-accounting overhead and scale behavior should be remeasured before replacing the wrapper-era numbers.

F. Scale and concurrency

We separately tested credential-token edge cases. The updated prototype denied all tested edge cases: credential A with token B, replaying a token with a second credential, binding a second active task in the same session, wrong token audience, wrong token actor, expired token, and revoked task state. We also tested safe-view drift checks: registry version drift, safe-view policy-version drift, and

view-definition hash mismatch were denied and require re-approval before execution.

The default seed contains 430 expenses, 240 employees, and 124 departments. A concurrency smoke test over the API completed 1, 5, and 20 concurrent sessions with zero failed sessions. At 20 concurrent sessions, query p50 was 82.896 ms and query p95 was 110.356 ms. This is an API-stack smoke test, not a throughput-capacity claim.

We also ran a synthetic scale sweep over 1k, 10k, and 100k scoped expense rows. Table VI reports the p50 latency for two representative query shapes. Raw PostgreSQL stays in the millisecond-to-tens-of-milliseconds range at 100k rows, and safe-view-only execution stays below 33 ms. Full SessionBound reaches 6.7–7.0 s at 100k rows. These results expose the scale-sensitive limitation of the PL/pgSQL reference runtime measured at the time and motivate the native hook path discussed above. They should not be read as a production-throughput result for the native hook/executor architecture.

G. Discussion of remaining gaps

The hardening results improve direct SQL-structure coverage and make the overhead source clearer, but several gaps remain. The PostgreSQL hook path now includes native executor accounting, but the disclosure unit is still demo-specific and the native path needs fresh overhead and scale measurement. Small-group aggregate inference remains a known limitation. The budget vector is operational and does not provide formal differential privacy.

XIV. Discussion

SessionBound separates task approval from database policy authoring. Managers and data owners approve work; the control plane translates that approval into a template, scope, TTL, budget vector, and signed token; the database runtime enforces the resulting session boundary. The runtime is not an LLM judge of task intent, and it does not require a business approver to write SQL predicates.

The runtime distinguishes filtering from denial. Scope predicates inside safe views may make out-of-scope rows disappear under normal SQL semantics. Attempts to leave the approved query surface–raw tables, denied fields, mutation, DDL, catalog escape, or blocked payload aggregation–are denied.

Budgets are authority accounting, not differential privacy. They measure how much task-approved information has been released and let receipts record how the approved authority was spent. SessionBound therefore composes with existing identity, delegation, RLS/VPD, policy, and data-catalog systems rather than replacing them.

XV. Related Work

Table VII summarizes the main distinction from existing mechanisms. The short answer to "why not just RLS plus short-lived credentials plus audit logs?" is that those mechanisms can implement important pieces of SessionBound, but they do not by themselves define the approved task as the unit of database execution. SessionBound's claim is the composition: a business approval is compiled into a short-lived session whose safe views, denied fields, row scope, SQL surface, budget vector, drift checks, and receipts are bound together.

A. Policy and delegation

ABAC, XACML, capabilities, and purpose-aware access control provide standing authorization abstractions over attributes, authority objects, and stated purposes [12], [13], [14], [15], [16], [17]. OAuth and authenticated delegation establish delegated authority [1], [2]. SessionBound uses a task token as a database-execution capability, but binds it to a credential, safe-view registry, budget vector, and receipt chain.

B. Agent and data-product governance

PAuth checks whether concrete tool operations are implied by a task description [3]; Data Product MCP governs discovery and access to enterprise data products through MCP [4], [5]. SessionBound assumes approval has already been reduced to structure and then governs the exploratory SQL session created from that approval.

C. Agent security and database enforcement

Prompt-injection work shows that agent prompts are not reliable security boundaries [18], [19], [20]; information-flow work explores stronger controls for agent actions [21], [22]. Database mechanisms such as RLS, Oracle VPD, and Oracle DDS enforce policy close to data [6], [7], [8], [23]. SessionBound uses database enforcement but makes the approved task session, not the standing identity or relation, the runtime object.

D. Query structure, auditing, and inference

SQLrand and SQLCHECK motivate parser-mediated query-structure validation [24], [25]. Query auditing, inference-control, differential privacy, and DLP study disclosure from answers or data movement [26], [27], [10], [11], [28], [29]. SessionBound's disclosure budgets are operational accounting for a delegated task, not a formal DP guarantee.

E. Relationship authorization

Zanzibar provides globally consistent relationship-based authorization checks [9]. Such checks can help decide whether a user or agent may request a task; SessionBound addresses the subsequent question of how the resulting database session should be scoped, budgeted, and audited.

TABLE IV
SECURITY-PROPERTY COMPARISON ACROSS MEASURED BASELINES. CREDENTIAL-TOKEN BINDING INCLUDES CREDENTIAL-ID MATCHING, ACTOR MATCHING, AUDIENCE VALIDATION, CROSS-CREDENTIAL REPLAY REJECTION, EXPIRATION, REVOCATION, AND SAME-SESSION REBIND REJECTION. SCHEMA DRIFT INVALIDATION CHECKS SAFE-VIEW REGISTRY VERSION, POLICY VERSION, AND VIEW-DEFINITION HASH AT BIND TIME.

Property	Raw	Role-only	Safe-view-only	RLS-only	SessionBound
Blocks writes	No	Yes	Yes	Yes	Yes
Blocks raw table access	No	No	Yes	No	Yes
Blocks denied fields	No	No	Yes	No	Yes
Enforces row scope	No	No	Yes	Yes	Yes
Query budget	No	No	No	No	Yes
Disclosure budget	No	No	No	No	Yes
Payload aggregation blocking	No	No	No	No	Yes
Credential-token binding	No	No	No	No	Yes
Receipts	No	No	No	No	Yes
Schema drift token invalidation	Not tested	Not tested	Not tested	Not tested	Yes

TABLE V
V3 OVERHEAD BREAKDOWN P50 LATENCY IN MILLISECONDS. THE RUN USED 10 WARMUP AND 100 MEASURED ITERATIONS PER PATTERN AND MODE. RLS-ONLY IS NOT SUPPORTED BY THE CURRENT PROTOTYPE AND IS REPORTED AS N/A.

Pattern	Raw	Role	Safe view	RLS	SB no receipt	SB no budget	SB full
SELECT	0.233	0.229	14.213	N/A	14.899	14.333	14.630
JOIN	0.182	0.171	18.092	N/A	18.497	18.238	18.581
GROUP BY	0.193	0.169	13.296	N/A	14.840	15.074	15.024
CTE	0.176	0.175	13.525	N/A	14.529	15.098	15.425
Window	0.351	0.347	13.828	N/A	15.089	14.852	15.854

TABLE VI
SYNTHETIC SCALE SWEEP P50 LATENCY IN MILLISECONDS. SAFE DENOTES SAFE-VIEW-ONLY EXECUTION; SB DENOTES FULL SESSIONBOUND.

Rows	Query	Raw	Safe	RLS	SB
1k	Agg.	0.41	0.60	0.39	84.41
1k	Top-k	0.40	0.53	0.43	82.44
10k	Agg.	2.70	3.76	2.38	688.66
10k	Top-k	2.54	3.32	2.39	661.62
100k	Agg.	14.77	29.96	15.08	6742.55
100k	Top-k	15.62	32.39	14.61	6965.09

XVI. FUTURE WORK

Future work falls into five directions. First, cross-database and lakehouse federation should preserve one approval, budget, and receipt chain across PostgreSQL, warehouses, vector stores, and lakehouse tables. Second, production SQL enforcement should move from the current wrapper/hook prototype toward planner- or executor-level integration that binds task state to resolved relations and accounts for returned business objects. Third, disclosure accounting should add sensitivity-aware budgets, aggregate leakage controls, minimum-group policies, and reconstruction-risk checks. Fourth, receipts can be strengthened with Merkle trees, append-only transparency logs, cryptographic accumulators, or external notarization. Fifth, safe-view authoring should become a developer workflow with linting, definition hashing, adversarial tests, and migration-driven token invalidation.

XVII. LIMITATIONS

SessionBound is a research prototype and reference architecture with the following limitations:

- SQL validation now includes AST-level API preflight and an experimental PostgreSQL parse/analyze hook path, but it is not production-grade planner/executor-level enforcement;
- the prototype targets a single PostgreSQL database and demo signing should be replaced with KMS/JWKS-backed signing;
- disclosure budgets are operational authority accounting, not formal differential privacy or complete semantic inference control;
- out-of-scope predicates over safe views return no rows through transparent scope filtering rather than producing an explicit denial;
- payload aggregation functions are conservatively blocked rather than governed by a per-template allowlist in the measured prototype;
- safe views and drift invalidation require production migration, review, and re-approval workflows;
- current performance results include microbenchmarks, overhead ablations, and a prior 1k/10k/100k synthetic scale sweep, but should not be generalized to production workloads or an optimized planner/executor-hook implementation;
- evaluated native paths persist allowed receipts and hook/API denial receipts through a same-database autonomous audit channel, including across agent transaction rollback; external audit retention, lifecycle cleanup, credential brokerage, and key management remain production infrastructure outside this demo;
- cross-database enforcement, malicious DBAs, and compromised hosts are out of scope.

TABLE VII
WHY SESSIONBOUND IS NOT JUST AN EXISTING ACCESS-CONTROL OR AUDITING MECHANISM. THE MECHANISMS LISTED HERE ARE COMPLEMENTARY IMPLEMENTATION BUILDING BLOCKS; SESSIONBOUND ADDS THE TASK-BOUND DATABASE-SESSION CONTRACT AROUND THEM.

Mechanism	What it provides	Why it is insufficient alone	SessionBound role
RLS / VPD / DDS	Predicates and context-aware policy near data	Strong data-plane enforcement, but not a task object that binds approval, TTL, safe-view versioning, SQL surface, cumulative budgets, and receipts	Uses database enforcement inside a task-session contract
ABAC / XACML / purpose policy	Expressive policy over attributes and purpose	Captures standing rules, but not one approved exploratory SQL workspace with per-session exposure accounting	Compiles approved task attributes into runtime state
JIT credentials / OAuth / delegation	Scoped or time-bounded authority	Expiry and delegation do not constrain arbitrary SQL shape or cumulative disclosure after a credential is issued	Binds credentials to a task token checked at execution
PAuth / Data Product MCP / Zanzibar	Task-operation checks, governed data products, and relationship permissions	Authorizes operations, products, or relationships rather than a multi-query database session	Supplies complementary inputs before issuing a task token
Audit / DP / DLP	After-the-fact records, statistical privacy, or leakage detection	Logs do not prevent release; DP has different statistical goals; DLP is not a task-session contract	Emits decision receipts and accounts budget before release

XVIII. CONCLUSION

SessionBound turns enterprise task approval into budgeted database sessions for AI agents. It addresses a gap between business governance and database enforcement: business users approve tasks, data platforms register safe views, agents generate SQL, and databases enforce the approved boundary.

SessionBound is not a replacement for OAuth, authenticated delegation, PAuth, Data Product MCP, Oracle DDS, Zanzibar, or differential privacy. It is a complementary architecture for enterprise-internal analytical work: temporary tasks that are too flexible for fixed SaaS screens and too sensitive for raw database access.

SessionBoundDB, our PostgreSQL runtime, demonstrates how signed task tokens, safe views, denied fields, row scope, query budgets, disclosure budgets, and receipts can be bound to one database session. The result is a practical contract between enterprise task approval and agentic database execution:

```
The agent can try. The database decides.
```

The performance results sharpen the implementation lesson. On the default dataset, full SessionBound adds little over safe-view-only execution, so the small-scale overhead is primarily safe-view and session-runtime overhead rather than receipt-chain insertion or budget updates. At 100k scoped rows, however, the PL/pgSQL reference runtime becomes multi-second while raw PostgreSQL and safe-view-only execution remain much faster; the hook-only microbenchmark shows that structural parse/analyze guarding is not the multi-second bottleneck. This does not change the core task-bound session model, but it does define the production engineering path: move from the wrapper-based reference runtime toward native executor accounting and optimized PostgreSQL integration.

REFERENCES

[1] D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, 2012. [Online]. Available: https://www.rfc-editor.org/info/rfc6749

[2] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, and A. Pentland, "Authenticated Delegation and Authorized AI Agents," 2025. [Online]. Available: https://arxiv.org/abs/2501.09674

[3] R. K. Sharma, L. Jiang, Z. Lin, and S. Chen, "PAuth - Precise Task-Scoped Authorization For Agents," 2026. [Online]. Available: https://arxiv.org/abs/2603.17170

[4] M. Tonnarelli, F. Scaramuzza, S. Harrer, and L. W. Dietz, "Data Product MCP: Chat with your Enterprise Data," 2026. [Online]. Available: https://arxiv.org/abs/2601.08687

[5] Model Context Protocol, *Model Context Protocol Specification*, 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-06-18

[6] PostgreSQL Global Development Group, *Row Security Policies*, PostgreSQL, 2026. [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[7] Oracle, *Using Oracle Virtual Private Database to Control Data Access*, Oracle, 2026. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/using-oracle-vpd-to-control-data-access.html

[8] ——, *What Is Oracle Deep Data Security*, Oracle, 2026. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/26/ddscg/what-is-oracle-deep-data-security.html

[9] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. L. Korn, A. Parmar, C. D. Richards, and M. Wang, "Zanzibar: Google's Consistent, Global Authorization System," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, 2019, pp. 33–46. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/pang

[10] N. R. Adam and J. C. Wortmann, "Security-Control Methods for Statistical Databases: A Comparative Study," *ACM Computing Surveys*, vol. 21, no. 4, pp. 515–556, 1989. [Online]. Available: https://doi.org/10.1145/76894.76895

[11] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, ser. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014, vol. 9. [Online]. Available: https://doi.org/10.1561/0400000042

[12] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," National Institute of Standards and Technology, Special Publication 800-162, 2019. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-162

[13] OASIS XACML Technical Committee, *eXtensible Access Control Markup Language (XACML) Version 3.0*, OASIS, 2013. [Online]. Available: https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[14] J. B. Dennis and E. C. Van Horn, "Programming Semantics for Multiprogrammed Computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966. [Online]. Available: https://doi.org/10.1145/365230.365252

[15] M. S. Miller, K.-P. Yee, and J. Shapiro, "Capability Myths Demolished," Johns Hopkins University Systems Research Laboratory, Tech. Rep. SRL2003-02, 2003. [Online]. Available: https://classpages.cselabs.umn.edu/Fall-2021/csci5271/papers/SRL2003-02.pdf

[16] R. Agrawal, J. Kiernan, R. Srikant, and Y. Xu, "Hippocratic Databases," in *Proceedings of the 28th International Conference on Very Large Data Bases*, 2002, pp. 143–154. [Online]. Available: https://www.vldb.org/conf/2002/S05P02.pdf

[17] J.-W. Byun and N. Li, "Purpose Based Access Control for Privacy Protection in Relational Database Systems," *The VLDB Journal*, vol. 17, no. 4, pp. 603–619, 2008. [Online]. Available: https://doi.org/10.1007/s00778-006-0023-0

[18] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023. [Online]. Available: https://doi.org/10.1145/3605764.3623985

[19] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, "Prompt Injection Attack against LLM-integrated Applications," 2023. [Online]. Available: https://arxiv.org/abs/2306.05499

[20] OWASP GenAI Security Project, *LLM01:2025 Prompt Injection*, OWASP, 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[21] A. C. Myers and B. Liskov, "A Decentralized Model for Information Flow Control," in *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles*, 1997, pp. 129–142. [Online]. Available: https://www.cs.cornell.edu/andru/papers/iflow-sosp97/paper.html

[22] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI Agents with Information-Flow Control," 2025. [Online]. Available: https://arxiv.org/abs/2505.23643

[23] Oracle, *Oracle Deep Data Security Technical Brief*, Oracle, 2026. [Online]. Available: https://www.oracle.com/a/ocom/docs/security/deep-data-security-technical-brief.pdf

[24] S. W. Boyd and A. D. Keromytis, "SQLrand: Preventing SQL Injection Attacks," in *Applied Cryptography and Network Security*. Springer, 2004, pp. 292–302. [Online]. Available: https://doi.org/10.1007/978-3-540-24852-1_21

[25] Z. Su and G. Wassermann, "The Essence of Command Injection Attacks in Web Applications," in *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2006, pp. 372–382. [Online]. Available: https://doi.org/10.1145/1111037.1111070

[26] J. Kleinberg, C. Papadimitriou, and P. Raghavan, "Auditing Boolean Attributes," in *Proceedings of the Nineteenth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2000. [Online]. Available: https://doi.org/10.1145/335168.335210

[27] S. U. Nabar, B. Marthi, K. Kenthapadi, N. Mishra, and R. Motwani, "Towards Robustness in Query Auditing," in *Proceedings of the 32nd International Conference on Very Large Data Bases*, 2006, pp. 151–162. [Online]. Available: https://www.vldb.org/conf/2006/p151-nabar.pdf

[28] S. Alneyadi, E. Sithirasenan, and V. Muthukkumarasamy, "A Survey on Data Leakage Prevention Systems," *Journal of Network and Computer Applications*, vol. 62, pp. 137–152, 2016. [Online]. Available: https://doi.org/10.1016/j.jnca.2016.01.008

[29] National Institute of Standards and Technology, *Data Loss Prevention*, NIST Computer Security Resource Center Glossary, 2026. [Online]. Available: https://csrc.nist.gov/glossary/term/data_loss_prevention